Day 94 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 4 reference

Watch: Nucleus Sampling: The Curious Case of Neural Text Degeneration (TechViz) | Language Model Evaluation and Perplexity (Machine Learning TV)

`day94_generated_text_quality.py`, segment by segment, in the order `main()` uses things. Day 93's generation and Day 91's Kneser-Ney model are imported and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = the classifier's features, one row per passage · w, b = its weights and bias · k = piece length for the copying measure

Segment 0 — Setup

Settings

CODE

```
N, PROMPT_LEN, GEN_LEN = 120, 32, 300
KS = (8, 12, 16, 24, 32)
BUCKETS, FOLDS = 4096, 5
L2, CLF_STEPS, CLF_LR = 1e-3, 500, 0.5
SPEAKER = re.compile(r"^[A-Z][A-Z' ]+:$")
```

- `re.compile(r"^[A-Z][A-Z' ]+:$")` — a compiled pattern: ^ and $ anchor it to the whole line, [A-Z']+ is one or more capitals, spaces or apostrophes.
- `KS = (8, 12, 16, 24, 32)` — piece lengths: short pieces are shared by chance, long ones mean copying.

Segment 1 — A reference generator

kn_generate: a counting model that writes

CODE

```
def kn_generate(kn, vocab, prompt_text, n, rng, top_p=None):
    text = prompt_text
    for _ in range(n):
        ctx = text[-(kn.n - 1):]
        p = np.array([kn.prob(ctx, c) for c in vocab])
        nxt = d93.sample_next(np.log(p)[None], rng, top_p=top_p)[0]
        text += vocab[nxt]
    return text[len(prompt_text):]
```

- `text[-(kn.n - 1):]` — the last n – 1 characters: all the context Kneser-Ney uses.
- `np.log(p)[None]` — Day 93's sampler takes scores, so the probabilities go in as their logarithms, as one row.

Segment 2 — The measurements

features and logistic_cv: a classifier from scratch

MATH

$p = 1 / (1 + e^{-(w \cdot x + b)})$; $loss = -mean[y \log p + (1 - y) \log(1 - p)] + (\lambda/2)|w|^2$; $dloss/dw = X^T(p - y)/n + \lambda w$

CODE

```
def features(texts):
    X = np.zeros((len(texts), BUCKETS))
    for r, t in enumerate(texts):
        for n in (1, 2, 3):
            for i in range(len(t) - n + 1):
                X[r, zlib.crc32(t[i:i + n].encode()) % BUCKETS] += 1
    return X / np.linalg.norm(X, axis=1, keepdims=True)
def logistic_cv(Xa, Xb, seed=0):
    X = np.vstack([Xa, Xb]); y = np.r_[np.zeros(len(Xa)), np.ones(len(Xb))]
    folds = np.random.RandomState(seed).permutation(len(y)) % FOLDS
    acc = []
    for f in range(FOLDS):
        tr, te = folds != f, folds == f
        w, b = np.zeros(X.shape[1]), 0.0
        for _ in range(CLF_STEPS):
            p = 1 / (1 + np.exp(-(X[tr] @ w + b)))
            g = p - y[tr]
            w -= CLF_LR * (X[tr].T @ g / tr.sum() + L2 * w); b -= CLF_LR * g.mean()
        acc.append(np.mean(((X[te] @ w + b) > 0) == y[te]))
    return np.mean(acc), np.std(acc)
```

- `zlib.crc32(t[i:i + n].encode()) % BUCKETS` — a fixed 32-bit hash of each piece, into one of 4,096 columns (Python's hash() changes per run).
- `np.random.RandomState(seed).permutation(len(y)) % FOLDS` — a random fold number 0-4 for every passage.
- `1 / (1 + np.exp(-(X[tr] @ w + b)))` — the logistic function: a probability that the passage is generated.
- `w -= CLF_LR * (X[tr].T @ g / tr.sum() + L2 * w)` — gradient descent on the mean log-loss, plus a small pull of the weights towards 0.

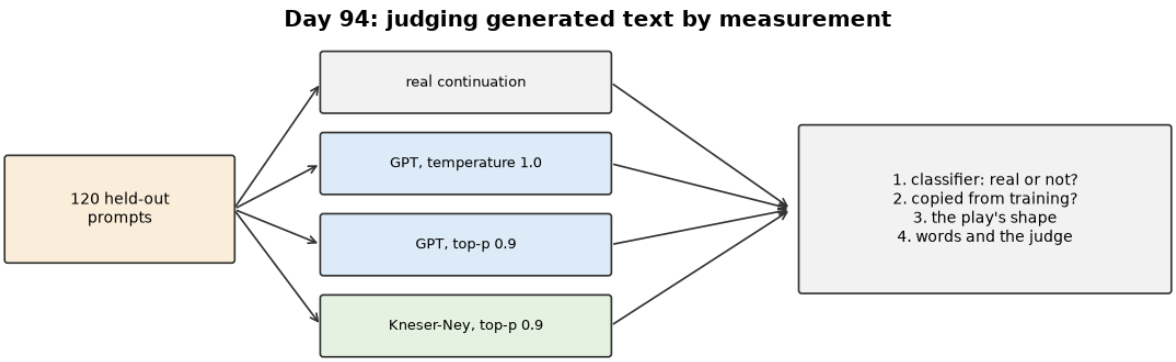
copy_share and shape_stats

CODE

```
def copy_share(texts, train_text, k):
    seen = {train_text[i:i + k] for i in range(len(train_text) - k + 1)}
    pieces = [t[i:i + k] for t in texts for i in range(len(t) - k + 1)]
    return np.mean([p in seen for p in pieces])
def shape_stats(texts):
    lines = [ln for t in texts for ln in t.split("\n")[1:-1] if ln.strip()]
    return np.mean([bool(SPEAKER.match(ln)) for ln in lines]), np.mean([len(ln) for ln in lines])
```

- `{train_text[i:i + k] for i in ...}` — a set comprehension: every k-piece of the training text, for fast membership tests.
- `p in seen` — set membership takes about the same time however big the set is.
- `t.split("\n")[1:-1]` — lines, without the first and last, which a passage may have cut in the middle.

SYSTEM DIAGRAM



Four kinds of passage from the same prompts, four tests.

Segment 3 — Experiments

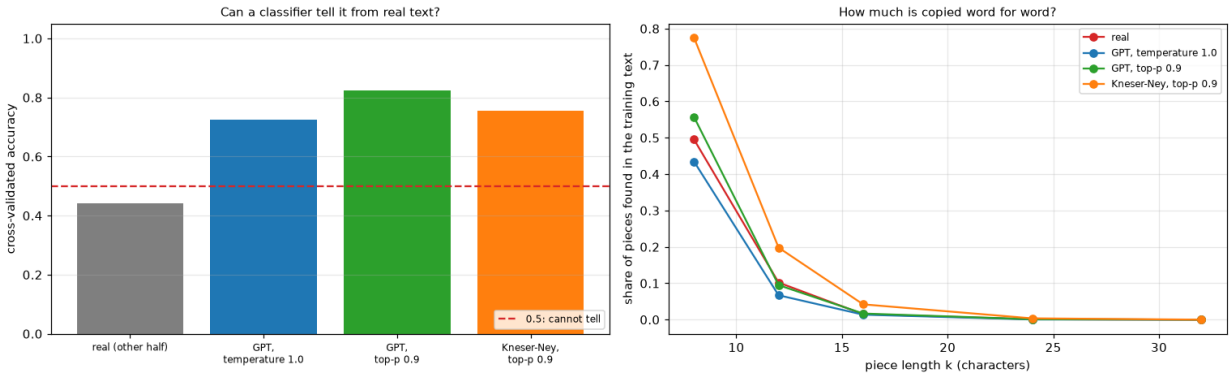
demo_classifier: real against each source

CODE

```
Xr = features(src["real"]); out = {}
print(f"\n{'real against':<26}{'accuracy':>10}{'spread over folds':>19}")
half = len(Xr) // 2
m, s = logistic_cv(Xr[:half], Xr[half:]); out["real (other half)"] = m
print(f"{'real (other half)':<26}{m:>10.3f}{s:>19.3f}")
for name in [k for k in src if k != "real"]:
    m, s = logistic_cv(Xr, features(src[name])); out[name] = m
    print(f"{name:<26}{m:>10.3f}{s:>19.3f}")
```

- `Xr[:half], Xr[half:]` — the control: real passages split in two, which no classifier should separate.
- `[k for k in src if k != "real"]` — every source except the real text.

REAL GRAPH



Real graph from the script: classifier accuracy, and copying by piece length.

Quick Syntax Glossary — new or reinforced constructs

- `zlib.crc32(b)` — a fixed 32-bit checksum, usable as a hash.
- `re.compile(pattern).match(s)` — does the start of s fit the pattern (whole line, with ^ and $)?
- `{expr for x in xs}` — a set comprehension.
- `x in a_set` — fast membership.

- `np.vstack([A, B])` — stack rows of two arrays.
- `a.permutation(n) % k` — random group labels 0..k-1.

End of Day 94 syntax reference. For the experiments and what they showed, see the Day 94 complete guide.